
Test plan (Khudsuno app)

1. Introduction

The *Khudsuno* mobile application ,through avatars and real time translation aims to remove communication barriers for the deaf and dumb members of the society and integrate AI models for sign language translation(PSL only).We are conducting a test plan below to outline all the techniques that will be followed and the approaches taken to test all functionalities as well and to outline the core responsibilities and levels the app maintains and can be expected to fulfill

2. Objectives

- Ensure accuracy in translation
- Validate avatar animations and their functionality
- online offline (apps behavior testing)
- usability, availability and security of the system.
- Overall app performance
- smooth integration verification (with external APIs and databases)

3. Scope

- **Functional Testing:** Gesture-to-text/speech translation, speech-to-sign translation, avatar accuracy.
- **Non-functional Testing:** Performance (speed, scalability), usability, and compatibility (across devices and platforms).
- **Security Testing:** User authentication, data privacy, secure communication between the app and cloud services.
- **Integration Testing:** Test the integration between the frontend and backend also with external APIs.
- **Regression Testing:** Ensure new changes do not affect existing functionality of *Khudsuno* mobile app.

4. Testing Levels

1. Unit Testing:

- **Objective:** Test individual components, such as the gesture recognition model, speech recognition model, avatar system, etc.

- **Tools:** Unit testing frameworks (e.g., **JUnit**, **pytest** for Python components).
- **Responsibility:** Development team.
- 2. **Integration Testing:**
 - **Objective:** Verify the interaction between integrated components, like speech recognition and gesture-to-text conversion, backend APIs, and avatar animations.
 - **Tools:** **Postman** for API testing, **Selenium** for UI testing.
 - **Responsibility:** QA team.
- 3. **System Testing:**
 - **Objective:** End-to-end testing of the entire system which ensures that the app is functioning as it was planned.
 - **Techniques:** Functional testing, usability testing, performance testing etc..
 - **Responsibility:** QA team.
- 4. **User Acceptance Testing (UAT):**
 - **Objective:** Ensure the app meets user expectations and requirements. This testing is done by the target user group (deaf/mute users and hearing users).
 - **Techniques:** Real-world testing with end-users.
 - **Responsibility:** User group, overseen by the product owner.

5. Testing Techniques

1. **Black Box Testing:**
 - Used for testing the gesture-to-text and speech-to-sign translation features without focusing on internal code logic.
 - Focused on input-output validation.
2. **White Box Testing:**
 - Used for unit tests, particularly for testing the algorithms in the AI models (e.g., gesture recognition, speech processing).
3. **Regression Testing:**
 - Regular testing after any updates to ensure that new features don't break existing functionality.
 - Automated testing scripts will be set up for continuous integration.
4. **Exploratory Testing:**
 - Focused on testing the app's usability and performance in real-world scenarios. Performed by testers without predefined scripts to discover unexpected bugs.
5. **Load and Stress Testing:**
 - Ensure the system can handle high volumes of requests (e.g., many users inputting gestures or speech simultaneously).
 - Tools: **Apache JMeter** for simulating load and stress conditions.
6. **Security Testing:**

- Test the user authentication mechanism (login, registration), data storage (encryption of sensitive data), and communication security (e.g., SSL/TLS encryption).

6. Test Environment

- **Devices:** Android and iOS smartphones and tablets.
- **Simulators:** Android Studio and Xcode simulators for mobile testing.
- **Testing Tools:** Postman, Selenium, JMeter, pytest.
- **Cloud Integration:** Ensure the app interacts correctly with cloud-based models and databases.

7. Test Cases (Examples)

Test Case	Test Scenario	Expected Result
Gesture-to-speech translation	Input a predefined gesture	App outputs the corresponding speech accurately
Speech-to-sign translation	Input a predefined phrase	Avatar accurately displays corresponding gestures
Offline mode functionality	Use the app without internet	Local processing happens, albeit slower
Avatar animation testing	Check avatar for smooth and correct gestures	Avatar movements should match the gestures perfectly
Login and registration	User enters credentials	User is logged in and redirected to dashboard
API integration	Test communication with external APIs (e.g., Google API)	Data is fetched correctly and processed as expected

Software Development Plan for Khudsuno

1. Project Overview

The *Khudsuno* mobile app facilitates communication between deaf/mute and hearing individuals through real-time translation of sign language gestures and speech using AI models and avatars. The app will support multiple modes (online and offline), ensuring communication is possible even without internet access.

Primary Objectives

- Develop a seamless PSL translation app with gesture-to-text and text-to-speech capabilities.
- Integrate avatars for PSL translation, ensuring accurate representation of sign language gestures.
- Ensure smooth functionality across different platforms (Android, ios).

Secondary Objectives

- **Compatibility across Devices:** Making sure the app runs smoothly on various mobile platforms and different devices.
- **User Interface Improvements:** Enhancing the app's UI/UX based on feedback from beta testing.
- **Scalability:** Preparing the app for potential future expansion to support other sign languages beyond PSL.

Key Functionalities

- PSL to text/audio conversion.
- Text/audio to PSL conversion using avatars.
- Offline mode for translation without internet.
- Secure user authentication and data privacy.

2. Development Methodology

- **Agile:** We will follow this methodology to ensure flexibility, iterative progress and continuous feedback. Each sprint will focus on delivering functional increments of the app
- **Scrum:** Daily Scrum meetings will help keep the team aligned and on track also.

3. Development Phases

1. Requirement Gathering & Analysis (2 weeks)

- Identify user needs through stakeholder meetings and research.
- Define clear requirements for the core functionalities (sign-to-speech, speech-to-sign, avatar creation, etc.).
- Tools: **Jira, Confluence** for documentation.

2. Design Phase (3 weeks)

- **UI/UX Design:** Create wireframes and mockups of the application interface using Figma.
- **System Architecture:** Design the apps architecture e.g., cloud services integration, local processing, database structures
- **Avatar Design:** Use **Blender** and **Unity** for avatar modeling and animation.

- Review designs with stakeholders.
- 3. **Development Phase (8 weeks)**
 - **Frontend Development** (React Native):
 - Develop the app's UI based on the approved designs.
 - Implement user interaction elements like gesture input and speech input.
 - **Backend Development** (Flask, Python, SQL):
 - Set up the server to handle requests for translation (gesture to speech and vice versa).
 - Implement APIs for real-time communication with external services (e.g., Microsoft Azure for cloud computing, Google APIs for speech).
 - **Avatar Integration:**
 - Animate avatars using **Unity** and integrate with the system to display sign language gestures.
 - **AI Model Integration:**
 - Train and integrate the gesture recognition models (CNN models) and speech recognition models.
 - Tools: **TensorFlow, PyTorch** for model development.
- 4. **Testing & Debugging Phase (4 weeks)**
 - Perform unit, integration, and system testing (as detailed in the test plan).
 - Bug fixing and refinement based on feedback from testers and stakeholders.
- 5. **Deployment & User Training (2 weeks)**
 - Deploy the app on **Google Play Store** and **Apple App Store**.
 - Prepare a user guide to help users understand how to use the app effectively.
 - Tools: **Google Firebase** for app distribution and analytics.
- 6. **Post-Deployment Maintenance & Updates (Ongoing)**
 - Release patches and updates based on user feedback and any new requirements.
 - Maintain cloud-based services and models, ensuring smooth functioning of online services.

Phase	Duration	Milestones
Requirement Gathering & Analysis	2 weeks	Finalized requirements document
Design Phase	3 weeks	UI/UX design and system architecture ready.
Development Phase	8 weeks	First working version of the app.
Testing & Debugging	4 weeks	Bug-free app ready for deployment.
Deployment & User Training	2 weeks	App live on app stores with user guides.

Post-Deployment Maintenance	1 week	Regular updates and patches based on feedback.
------------------------------------	--------	--

4. Roles and Responsibilities

Role	Responsibility
Project Manager	Overseeing the project, ensuring deadlines are met
Frontend Developers	Developing the mobile app interface using React Native
Backend Developers	Implementing the server-side logic and API integrations
AI Model Engineers	Training and implementing the AI models for gesture and speech recognition
Avatar Designers	Creating and animating avatars using Blender and Unity
QA Engineers	Testing the app at different levels and providing feedback
System Administrators	Managing cloud services (Azure, Firebase)
UI/UX Designers	Designing the app interface and user experience

5. Timeline & Milestones

Phase	Duration	Milestone
Requirement Gathering & Analysis	2 weeks	Finalized requirements document
Design Phase	3 weeks	UI/UX and system architecture ready
Development Phase	8 weeks	First working version of the app
Testing & Debugging	4 weeks	Bug-free version of the app ready
Deployment & User Training	2 weeks	App live on app stores
Post-Deployment Maintenance	Ongoing	Regular updates and patches

6. Tools & Technologies

- **Frontend:** React Native, Figma for UI design.
- **Backend:** Flask, SQL for database management.

- **AI Models:** TensorFlow, PyTorch, Opencv for gesture and speech recognition.
- **Avatar Development:** Blender for avatar design and animations.
- **Cloud:** Microsoft Azure

SRS:

Functional Requirements:

The app should be compatible on all devices.

The users must be able to update their profile.

The system should be able to handle errors.

Avatar customization.

The users must be able to view the avatar.

The system must capture the input from users.

The system must handle common PSL gestures.

User must be able to login.

The system must process and translate the sign language.

The app must be responsive to different screen sizes.

The system must be able to handle errors.

Non-Functional Requirements:

The system must respond within a certain time frame.

The system must be easy to navigate.

The system must ensure correct authentication.

The system must ensure constant availability (upto 80%).

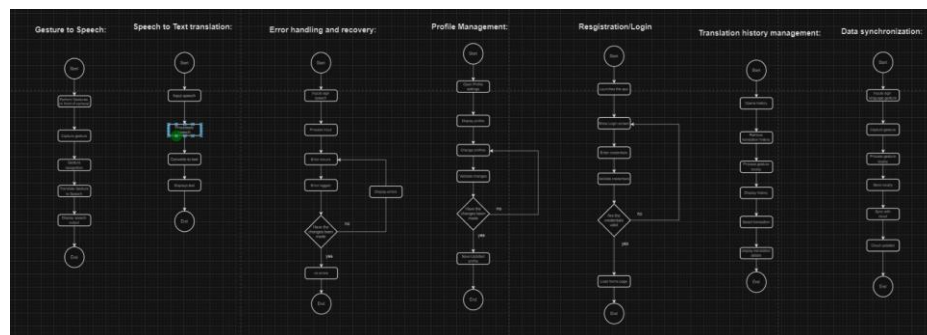
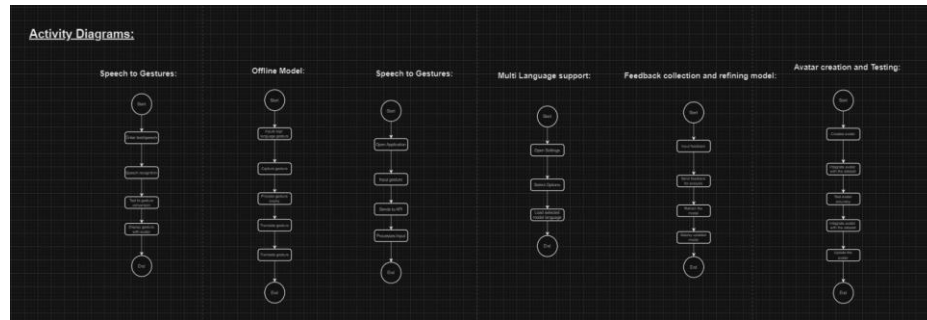
The system must be able to work in an offline mode.

The system must provide monitoring tools to detect errors.

The avatar must be integrated correctly.

The system must adhere to privacy policies.

1 Activity Diagrams



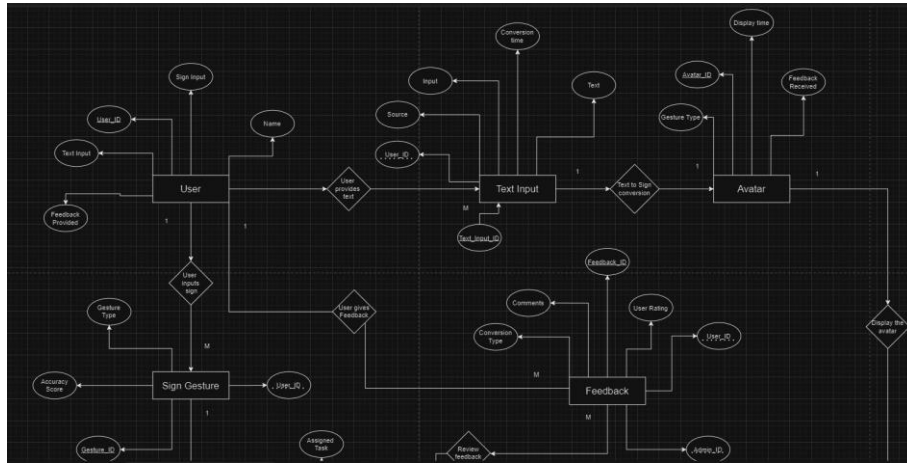


Figure 3: ER Diagram Part 1

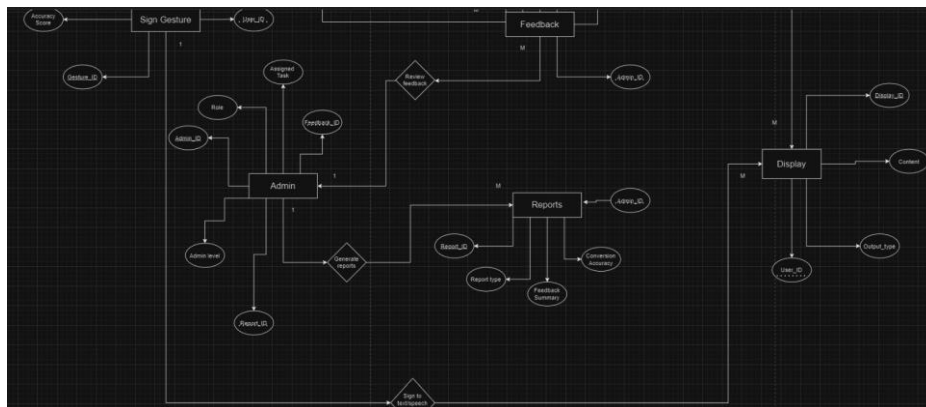
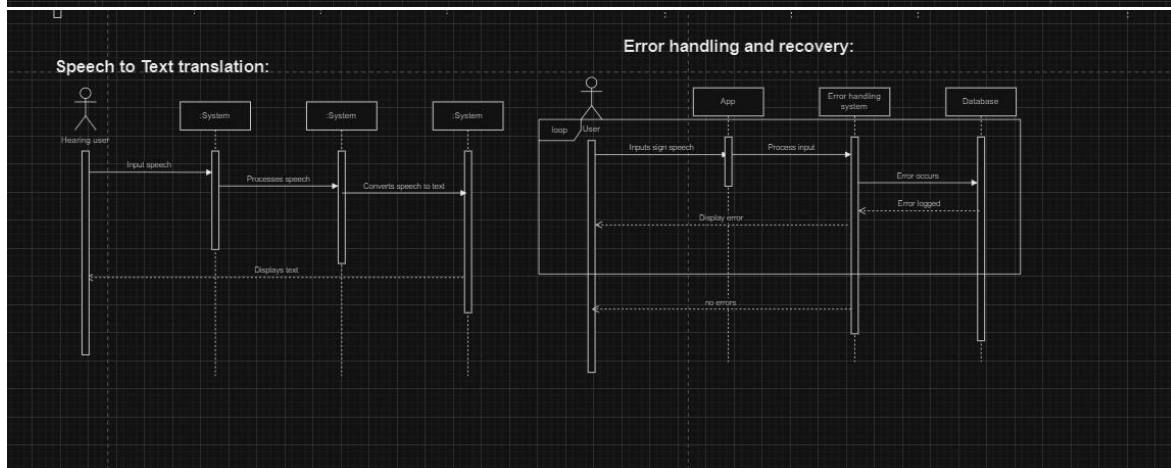
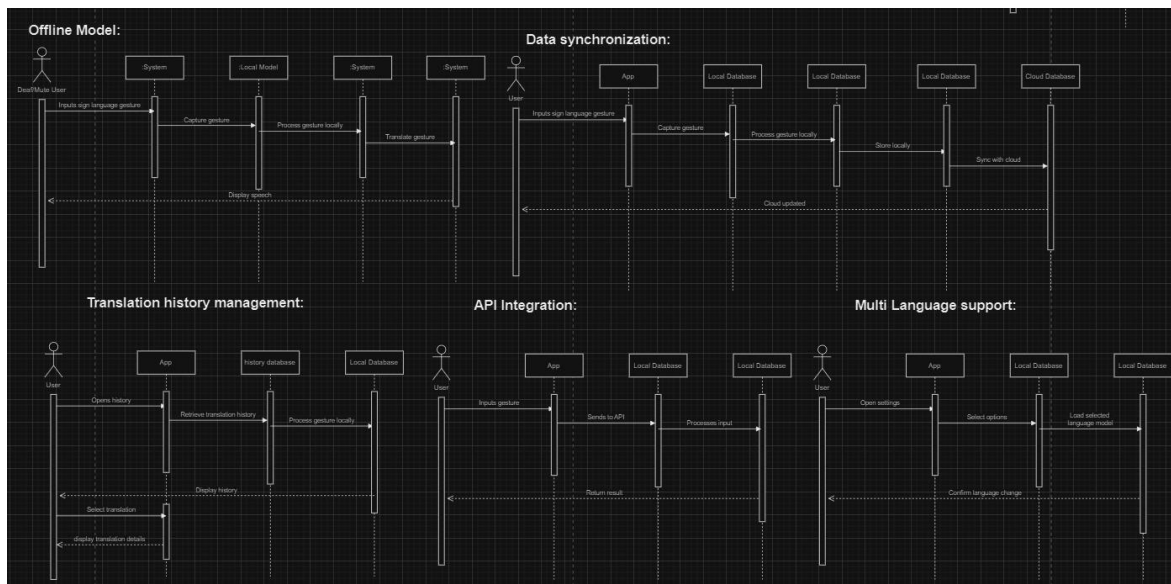
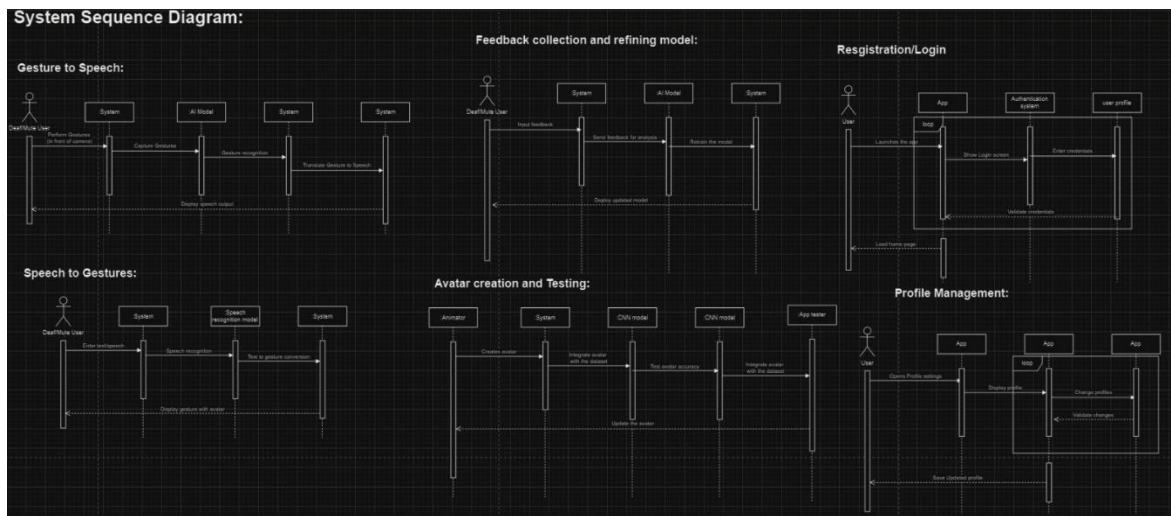


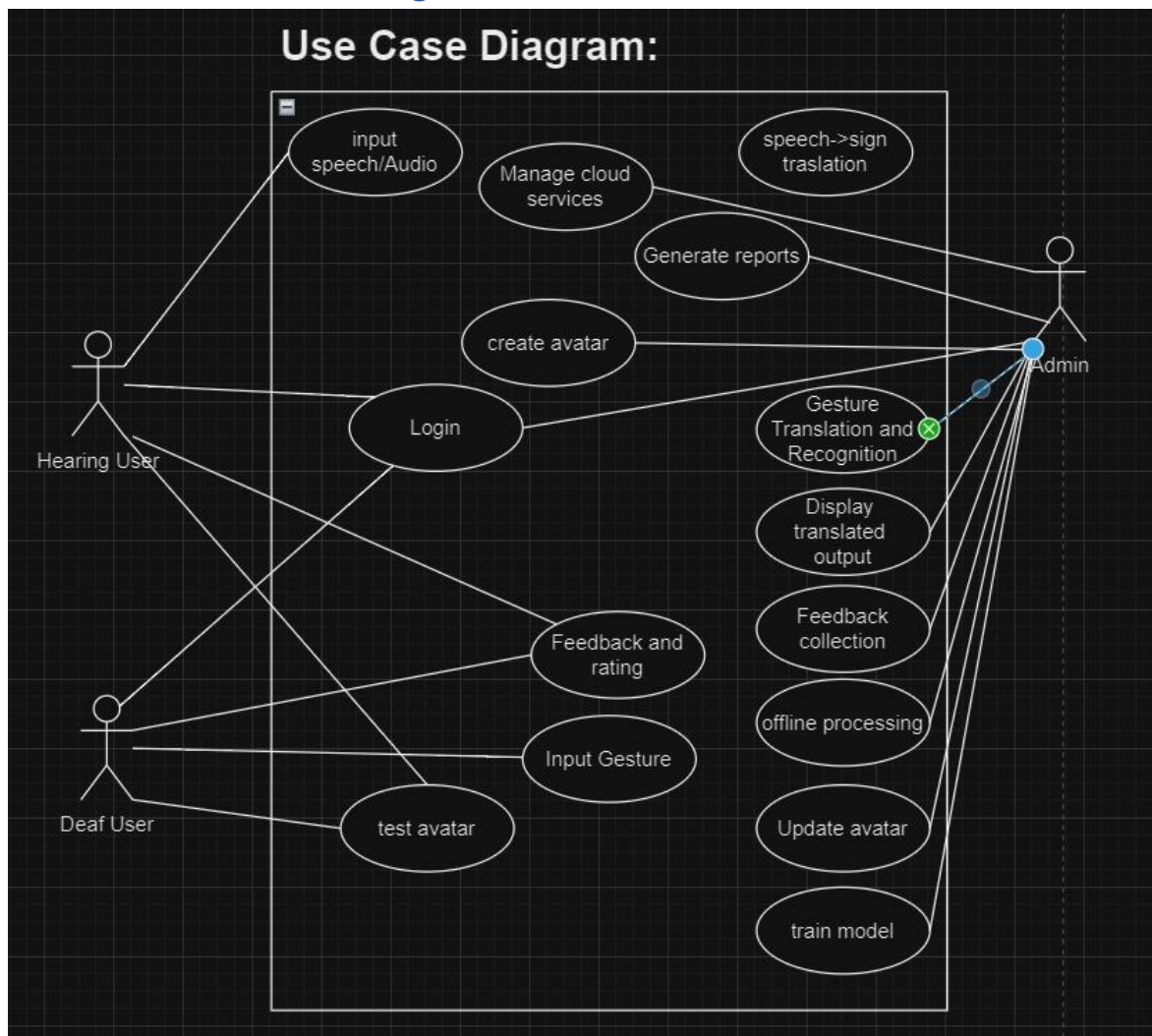
Figure 4: ER Diagram Part 2

3 SSD Diagrams



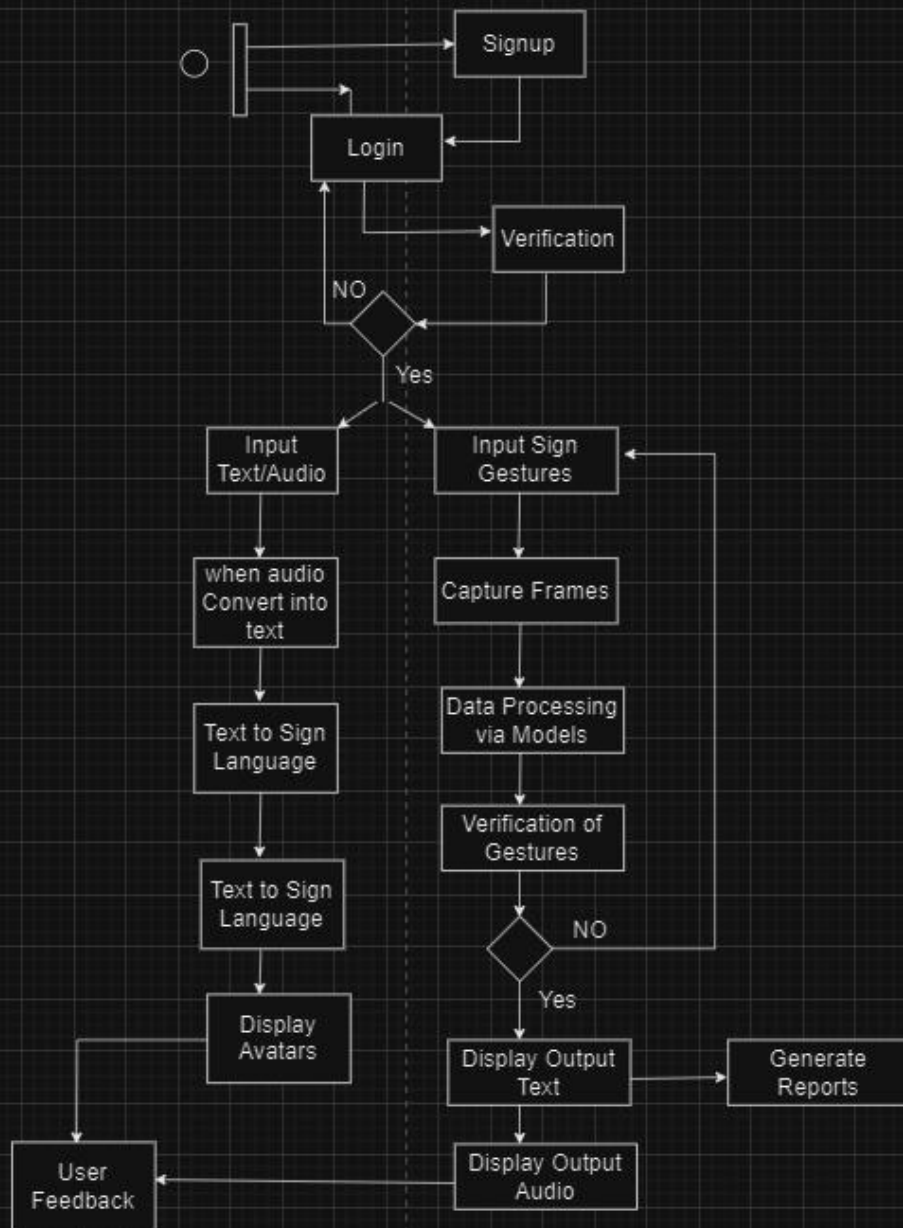
4

Use Case diagram

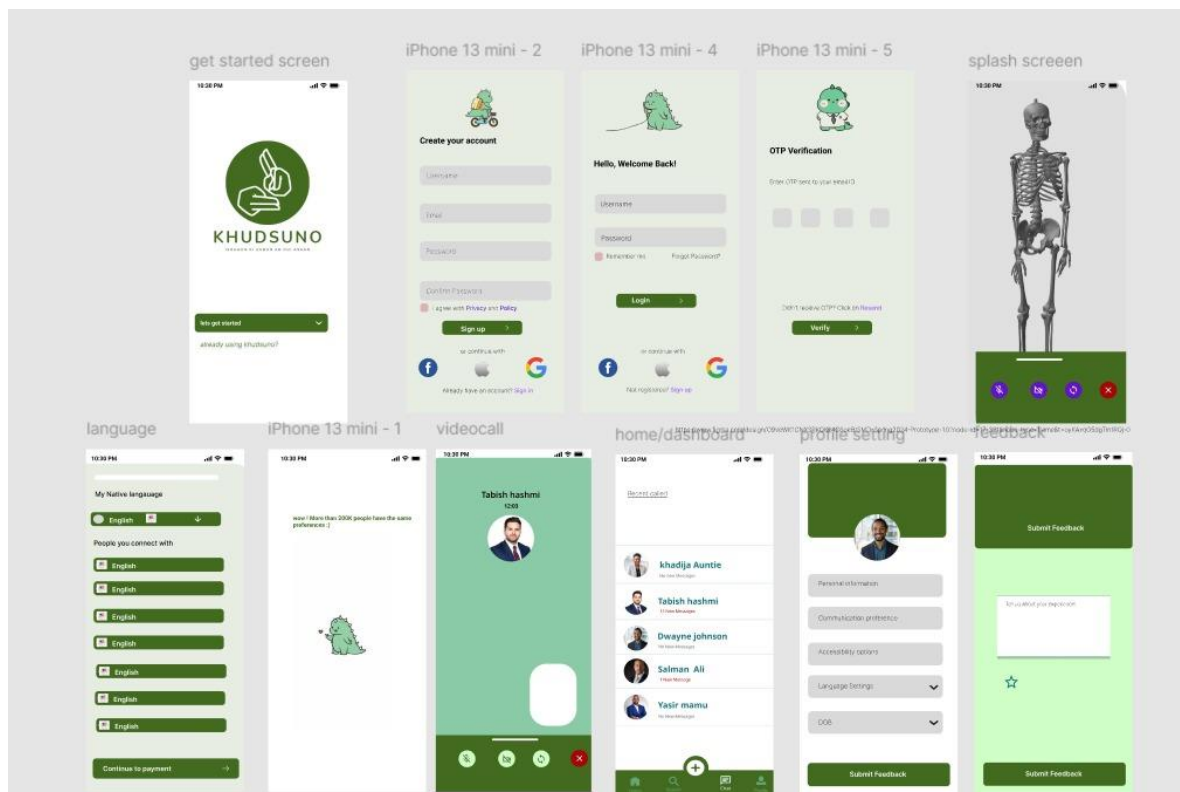


5 DataFlow diagram

Dataflow Diagram:



6 Figma Design



Links:

<https://www.figma.com/design/iXDkF0tADgFtvXsEKhDunH/Figma-basics?node-id=1669-162202&node-type=canvas&t=MhWnUCy1N92X7LaI-0>

<https://www.canva.com/design/DAGSTat7JKo/tQTyVYD7C-fxKOZKP1exEw/edit>

https://app.diagrams.net/#G1y5p6mXgfGgrx7rGZAG_NtkkhBCiWdw3d#%7B%22pageId%22%3A%22C5RBS43oDa-KdzZeNtuy%22%7D